

Julia Sets with SIMD

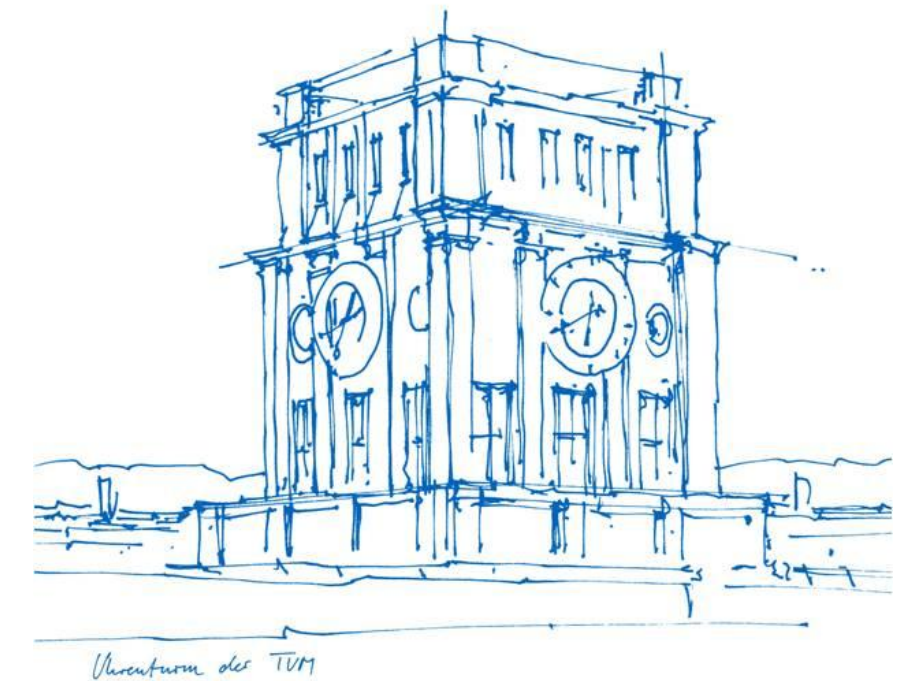
Implementation, Optimization and Performance Analysis in C

Technical University of Munich

IN0005 Grundlagenpraktikum: Rechnerarchitektur

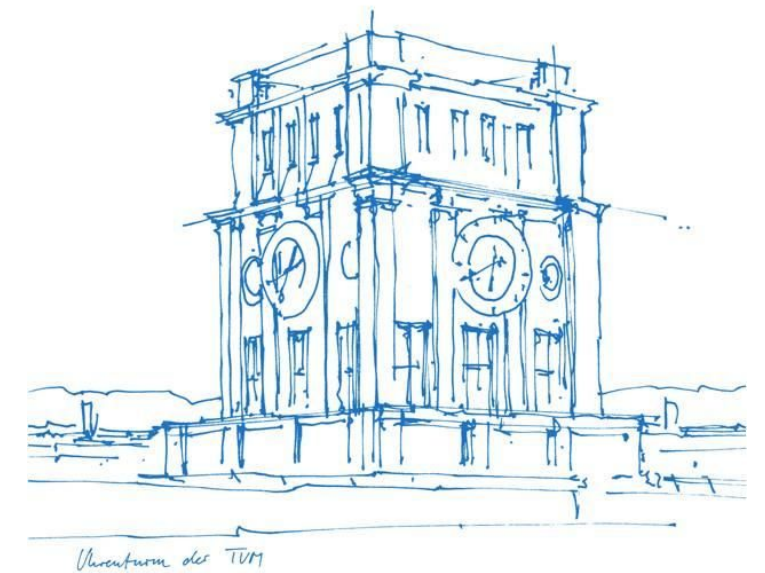
Chair of Computer Architecture and Parallel Systems

Erdeniz Taşlıçukur · Arda Taş · Noah Schneider



Agenda

- Problem Statement and Mathematical Foundation
- Scalar Reference and Baseline SIMD Design
- Correctness Validation and Edge Cases
- Benchmarking methodology and Environment
- Optimization Stages: Measurements and Trade-offs
- Final Performance Summary and Outlook



1 Introduction

$$z_0 = p, z_{i+1} = z_i^2 + c \ (i \geq 0, c, p \in \mathbb{C}) \cdot \text{let } z_i = a + bi, c = x + yi$$

1 Introduction

$$z_0 = p, z_{i+1} = z_i^2 + c \ (i \geq 0, c, p \in \mathbb{C}) \cdot \text{let } z_i = a + bi, c = x + yi$$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

1 Introduction

$$z_0 = p, z_{i+1} = z_i^2 + c \ (i \geq 0, c, p \in \mathbb{C}) \cdot \text{let } z_i = a + bi, c = x + yi$$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

Real part: $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

1 Introduction

$z_0 = p, z_{i+1} = z_i^2 + c \ (i \geq 0, c, p \in \mathbb{C})$ · let $z_i = a + bi, c = x + yi$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

Real part: $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

2 Iteration Update

The real and imaginary parts become their own update step:

```
vec_next.a = a2 - b2 + x; vec_next.b = 2ab + y
```

1 Introduction

$z_0 = p, z_{i+1} = z_i^2 + c \ (i \geq 0, c, p \in \mathbb{C})$ · let $z_i = a + bi, c = x + yi$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

Real part: $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

2 Iteration Update

The real and imaginary parts become their own update step:

$$\text{vec_next.a} = a^2 - b^2 + x; \text{vec_next.b} = 2ab + y$$

3 Escape Condition

A point escapes when $|z_i| > 2$, i.e. $\sqrt{a^2+b^2} > 2$ for a $|c| \leq 2$. To avoid the square root calculation, we check the squared magnitude instead:

$$\text{magnitude_check} = (\text{vec_next.a})^2 + (\text{vec_next.b})^2 > 4$$



Scalar Reference: `julia_v1()`

Pixels are processed one at a time by `julia_iterations()`.



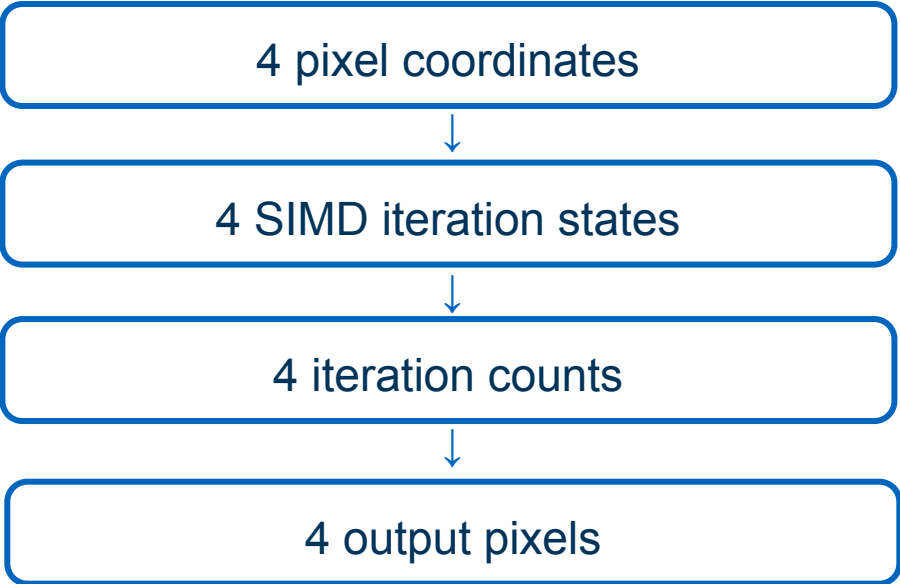
Baseline Four-Pixel SIMD Design

Four neighbouring pixels are processed using one sequence of vector instructions.

Example: `__m128 z_real`

Lane 0	Lane 1	Lane 2	Lane 3
zr_0	zr_1	zr_2	zr_3
Pixel x	Pixel x+1	Pixel x+2	Pixel x+3
32-bit float	32-bit float	32-bit float	32-bit float

`z_imag`, `counts` and `active` use separate registers with the same lane-to-pixel mapping.



The same arithmetic instruction operates on all four lanes simultaneously.



SIMD Update Step

Input: four complex states

__m128 z_real

Lane 0	Lane 1	Lane 2	Lane 3
zr_0	zr_1	zr_2	zr_3
Pixel x	Pixel x+1	Pixel x+2	Pixel x+3

__m128 z_imag

Lane 0	Lane 1	Lane 2	Lane 3
zi_0	zi_1	zi_2	zi_3
Pixel x	Pixel x+1	Pixel x+2	Pixel x+3

↓ both input registers feed both calculation branches ↓

One lane-wise Julia update

Real component

$\text{real_squared} = z_real * z_real$
 $\text{imag_squared} = z_imag * z_imag$
 $\text{next_real} = \text{real_squared} - \text{imag_squared} + c_real$
Broadcast constant: $c_real = [cr, cr, cr, cr]$

`_mm_mul_ps`

`_mm_sub_ps`

`_mm_add_ps`

Imaginary component

$\text{real_imag} = z_real * z_imag$

 $\text{next_imag} = 2 * \text{real_imag} + c_imag$
Broadcast constant: $c_imag = [ci, ci, ci, ci]$

`_mm_mul_ps`

`_mm_add_ps`

Output: four updated complex states

__m128 next_real

Lane 0	Lane 1	Lane 2	Lane 3
zr_next_0	zr_next_1	zr_next_2	zr_next_3

__m128 next_imag

Lane 0	Lane 1	Lane 2	Lane 3
zi_next_0	zi_next_1	zi_next_2	zi_next_3

SIMD Lane Divergence



128-bit Magnitude register (__m128)

Lane 0 Pixel (x, y)	Lane 1 Pixel (x+1, y)	Lane 2 Pixel (x+2, y)	Lane 3 Pixel (x+3, y)
------------------------	--------------------------	--------------------------	--------------------------

sIMD state

Previous active state

$zr^2 + z\bar{r}^2$

inside (≤ 4)

New active state

Iteration count

Lane 0	Lane 1	Lane 2	Lane 3
0xFFFFFFFF	0xFFFFFFFF	0x00000000	0xFFFFFFFF
3.2	4.6	1.9	5.1
0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000
0xFFFFFFFF	0x00000000	0x00000000	0x00000000
8	7	4	7

This also shows why our active state must be **persistent** : If a lane overflows back into the radius, it must remain.

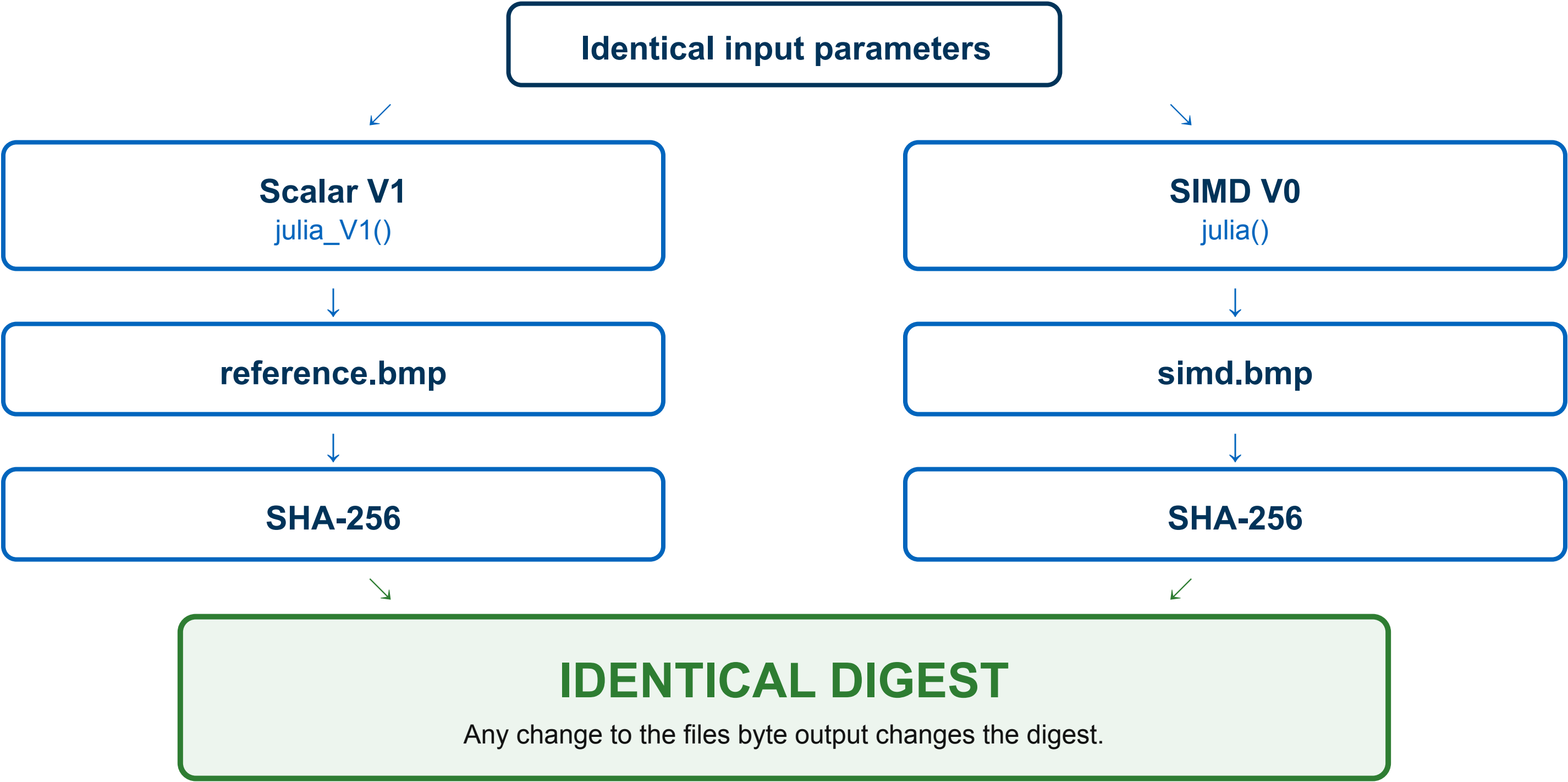
Tail case: the remaining width % 4 pixels use the scalar path.

```
inside = _mm_cmple_ps(magnitude_squared, four_v);
active = _mm_and_ps(active, inside);

if (_mm_movemask_ps(active) == 0)
    break;
```

inside: result of the current escape test.
active: persistent state across all iterations.
0xFFFFFFFF: active lane, all bits set.
0x00000000: inactive lane.
movemask: extracts most significant bit per lane;
zero means all pixels escaped.

Correctness



Same iteration
 $z_{i+1} = z_i^2 + c$

Same escape condition
 $|z_i|^2 \leq 4$

Same iteration limit
 n

Same pixel conversion
grayscale and color

Correctness



7 / 7 matched

0 mismatches

✓ **Default grayscale** Standard Julia calculation

✓ **SIMD tail, width 801** Final pixel processed scalar

✓ **Escaped lanes, $c = 1 + 1i$** Lanes stop at different iterations

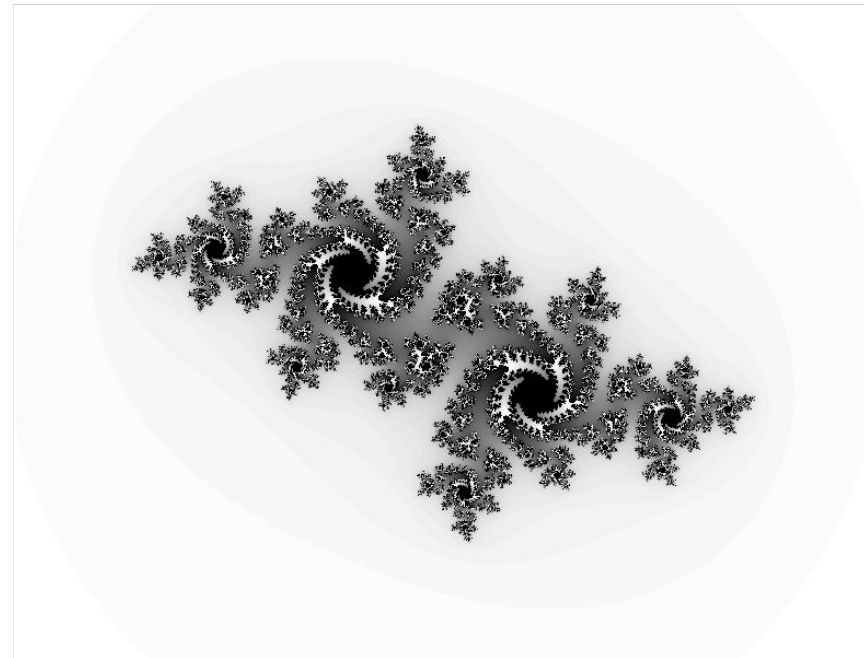
✓ **$n = 0$** Loop skipped; black output

✓ **Default color** Vectorized color conversion

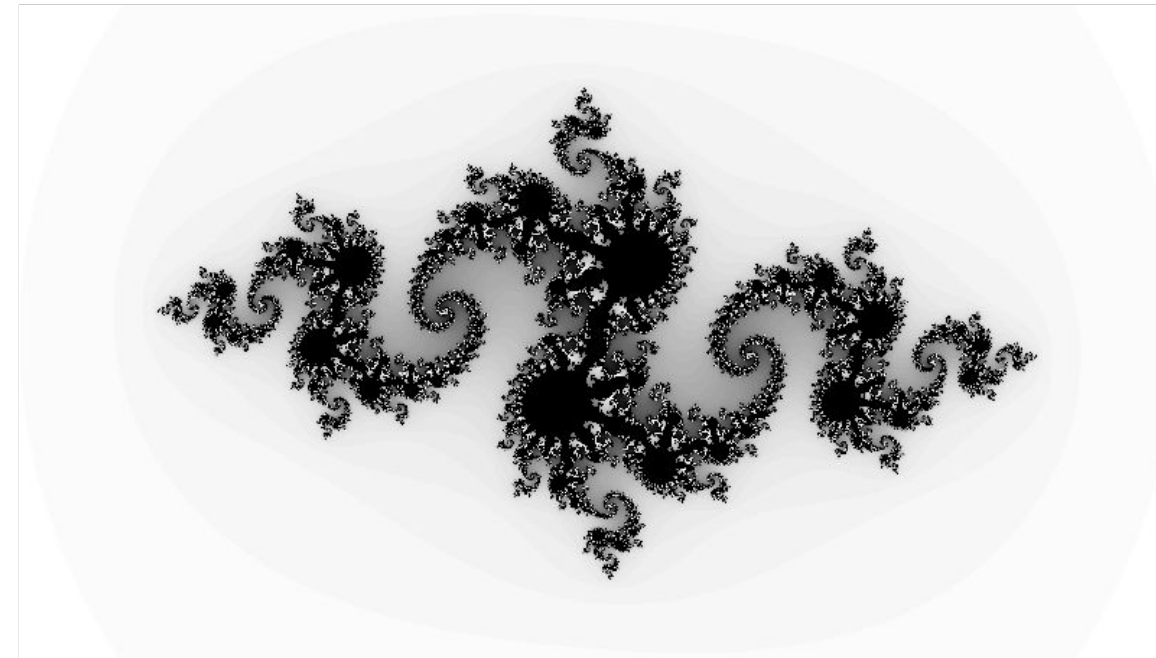
✓ **High iteration limit** Correct termination at n

✓ **Positive BMP height** Bottom-up row order

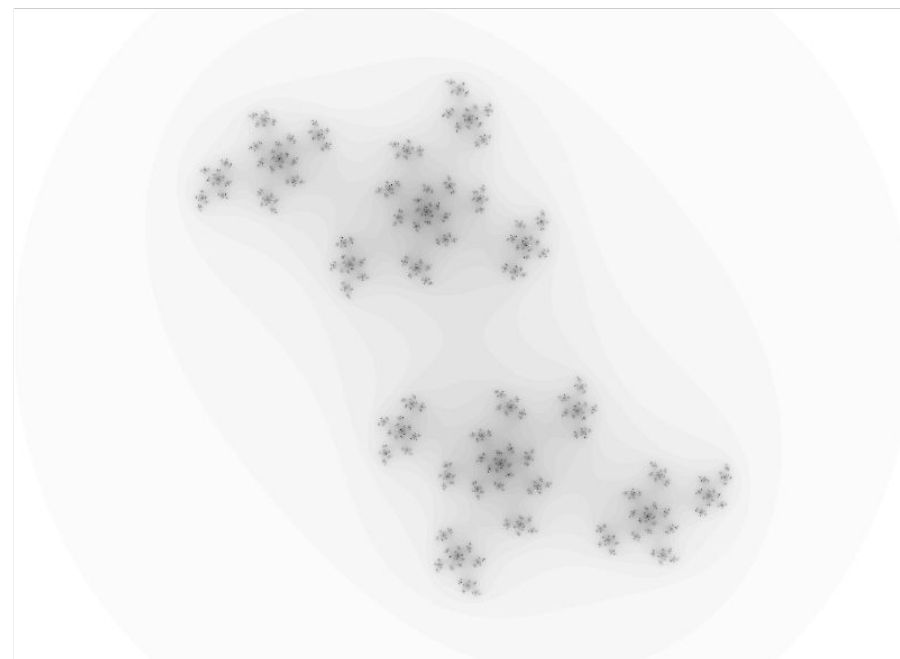
Outputs Produced by Different c Values



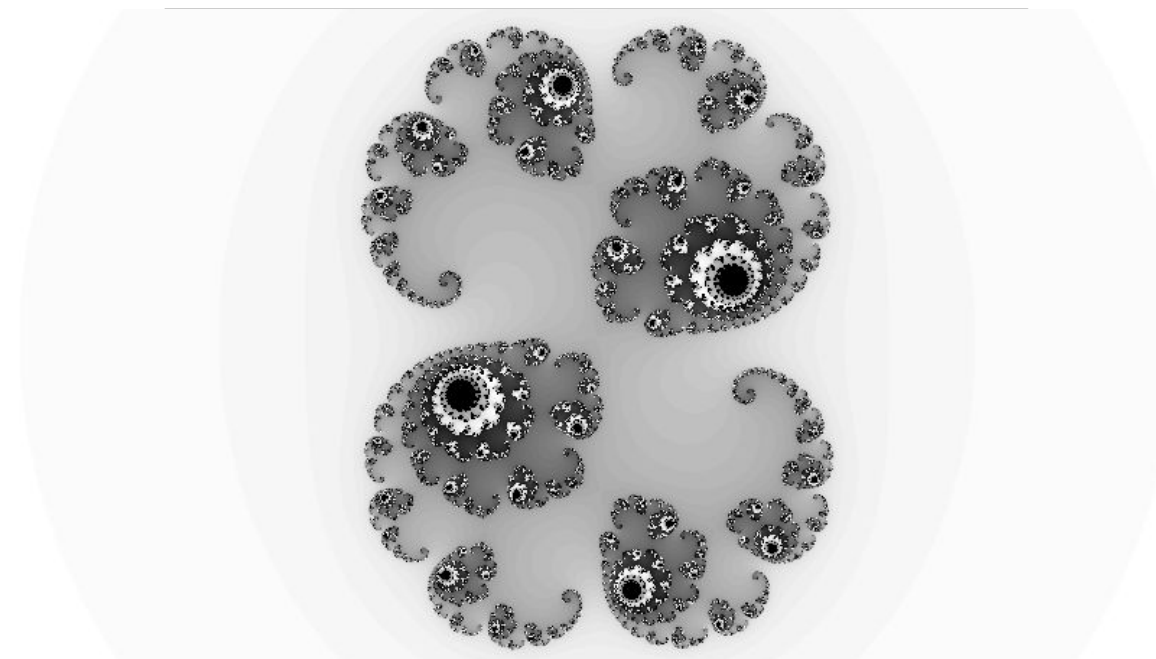
$$c = -0.5125 + 0.5213i$$



$$c = -0.8 + 0.156i$$



$$c = 0.123 + 0.745i$$



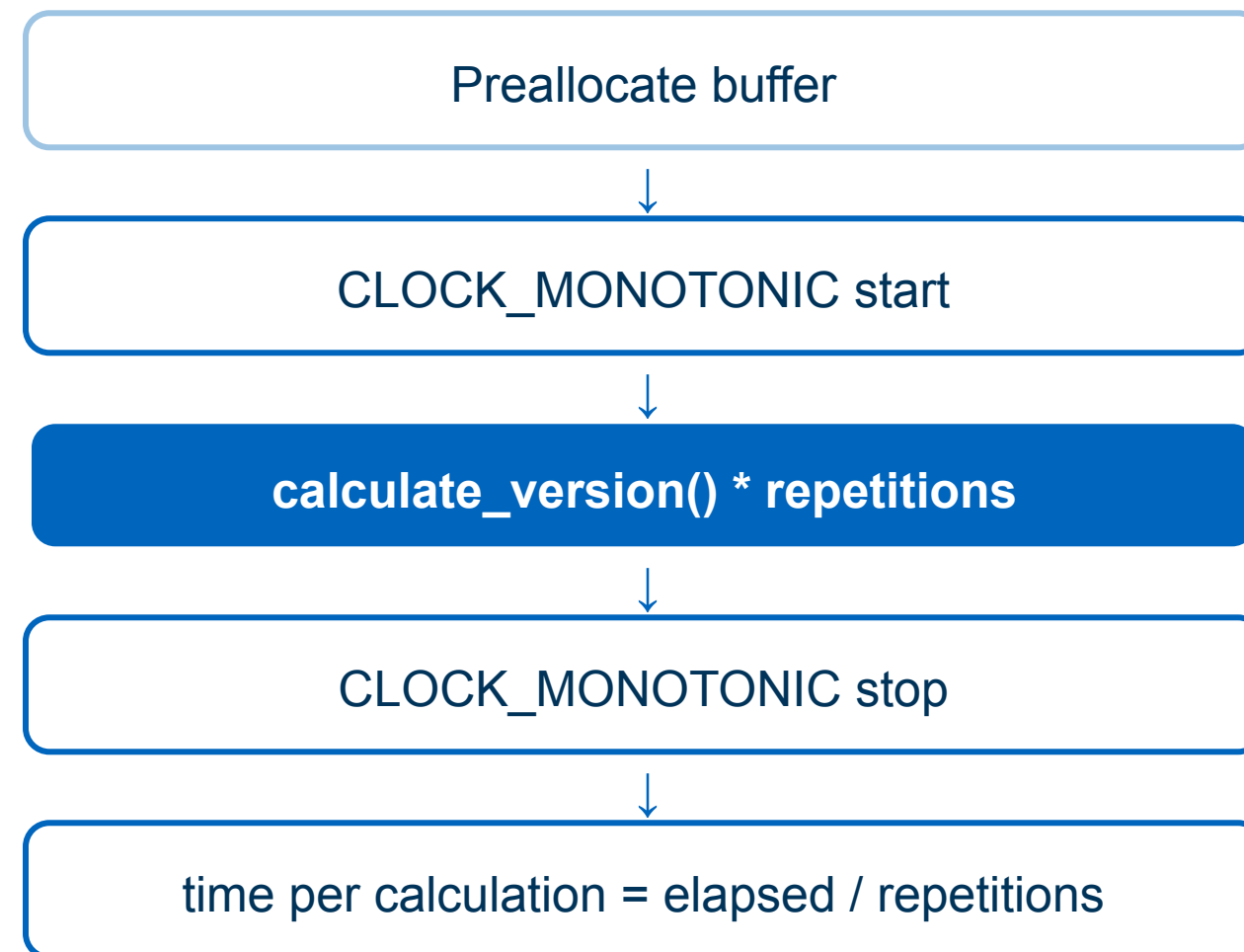
$$c = 0.285 + 0.01i$$

800 x 600; n = 100; res = 0.005; start = $-2 + 1.5i$

Benchmarking Methodology and Environment



All numbers on the following slides were measured the same way, on the same machine.



BMP writing excluded

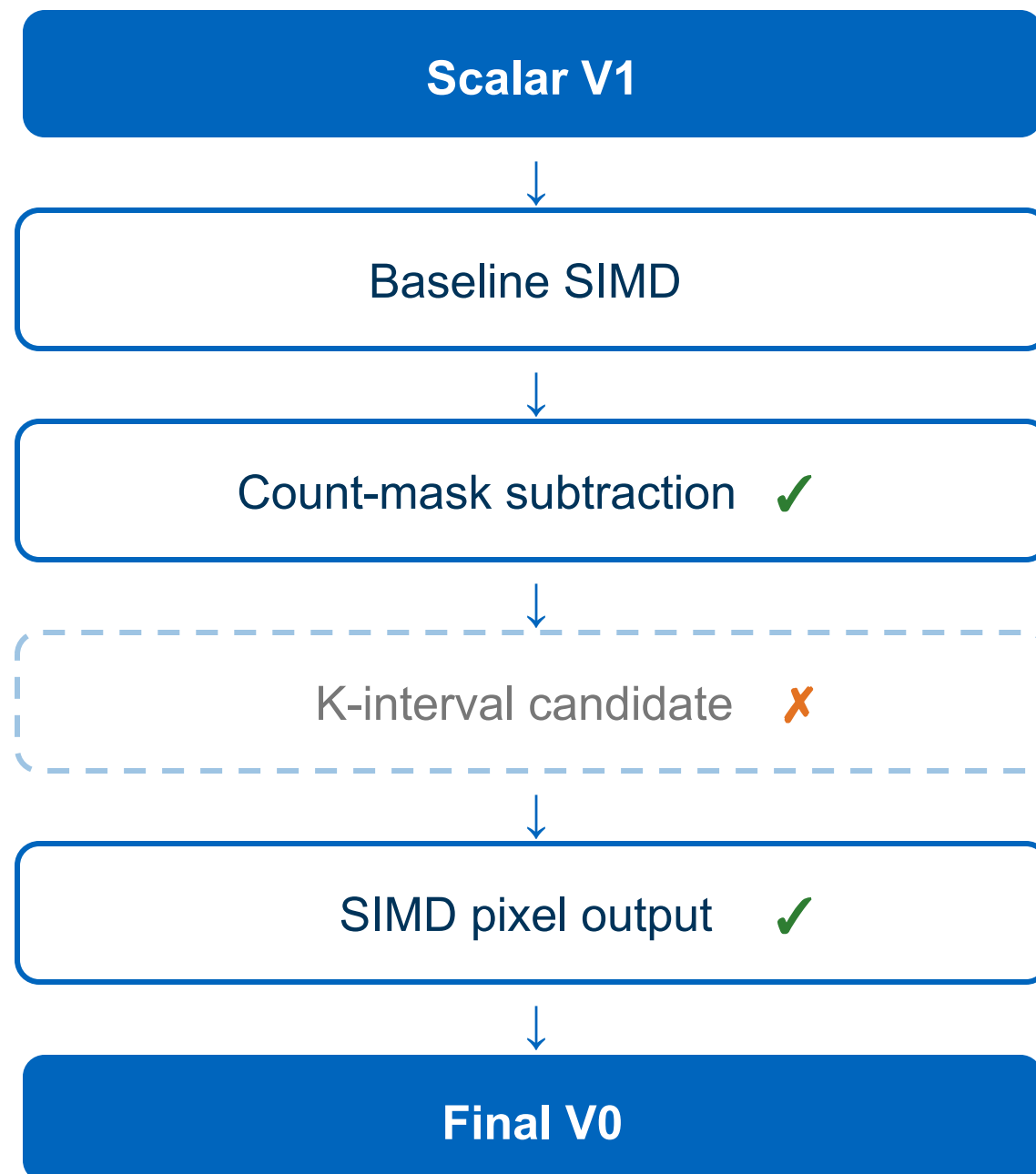
Environment

AMD Ryzen 9 5950X - x86-64

GCC 14.2; C17

-O3 -msse4.2

Debian 13.5 under WSL2



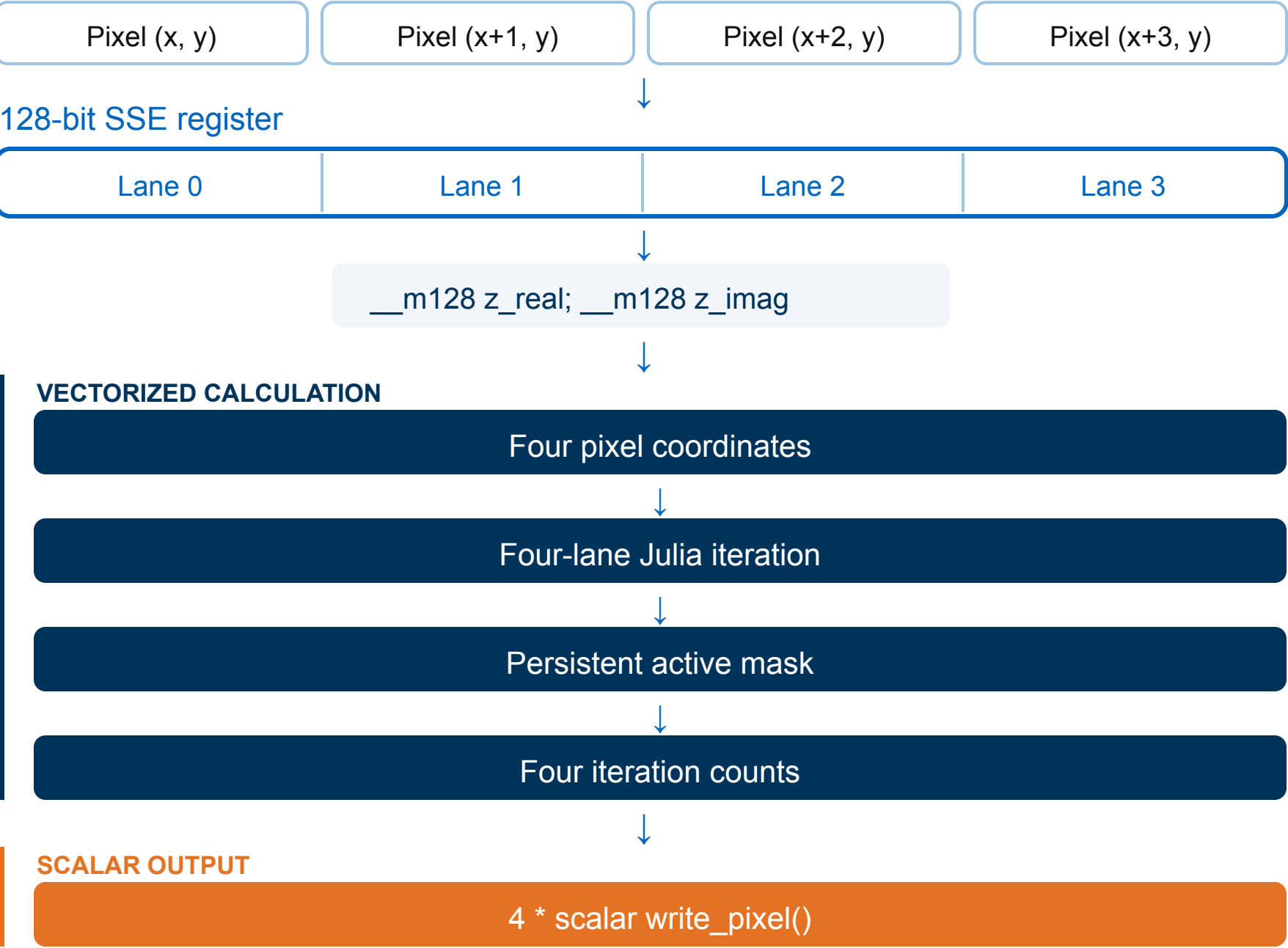
Chronology

How our final SIMD version developed throughout the project phase.

Methodology

Optimization targets were selected using profiling, not only theoretical expectations and verified for correctness and benchmarked against the previous version.

Baseline SIMD



Initial SIMD Result

V1 Scalar Reference



Baseline SIMD



Runtime [ms]

3.01x speedup

Approximately 75% of the theoretical four-lane SIMD limit

800 x 600 pixels - Grayscale - n = 100 - Average of 100 runs

Counter Update: AND Then ADD



Input registers

__m128 active

Lane 0	Lane 1	Lane 2	Lane 3
0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000
active	inactive	active	inactive

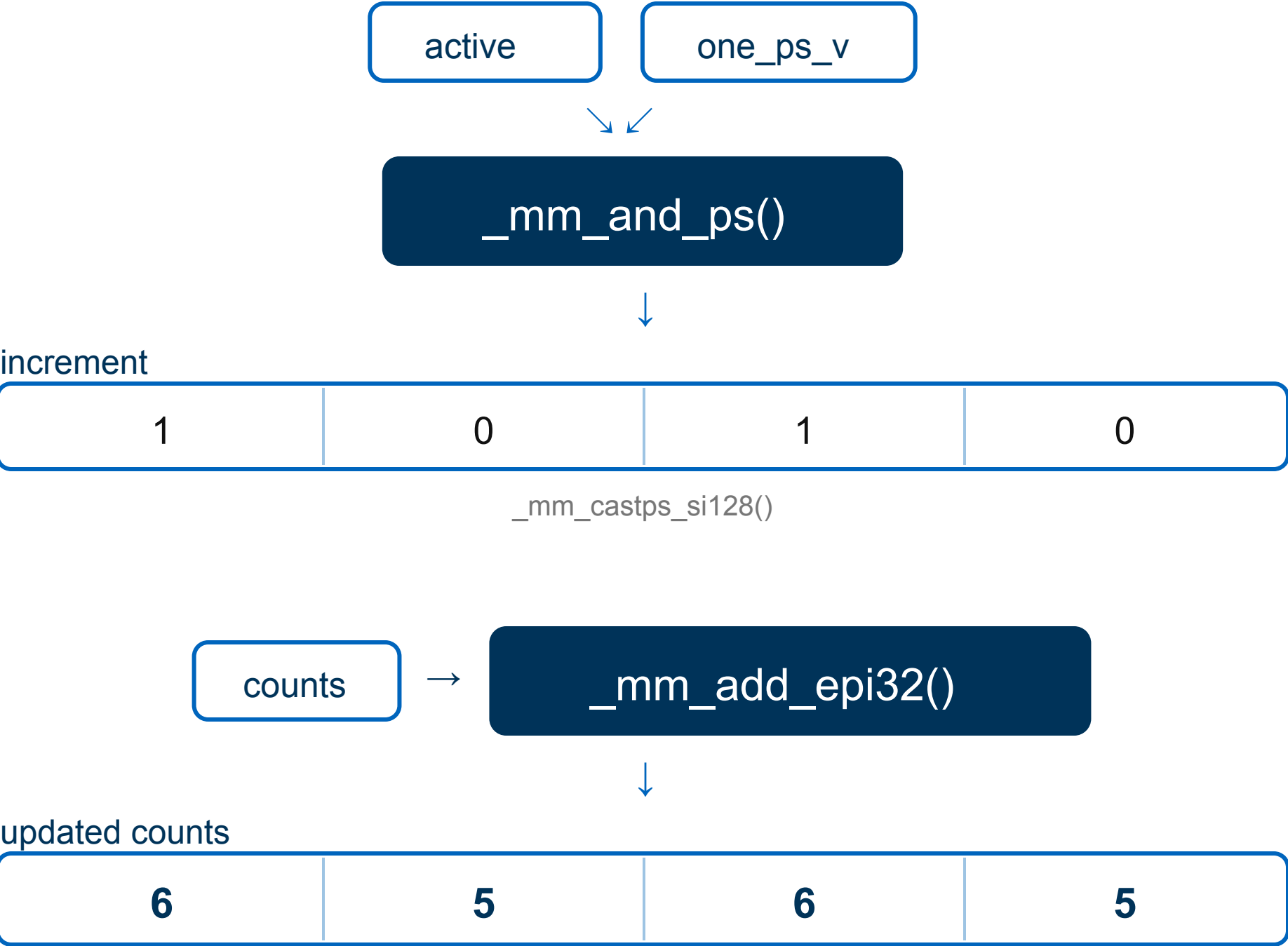
__m128 one_ps_v

Lane 0	Lane 1	Lane 2	Lane 3
0x00000001	0x00000001	0x00000001	0x00000001

__m128i counts

Lane 0	Lane 1	Lane 2	Lane 3
5	5	5	5

Dependent operations



Optimized Counter Update: Subtract the Mask

The same bits, interpreted as int32

__m128 active

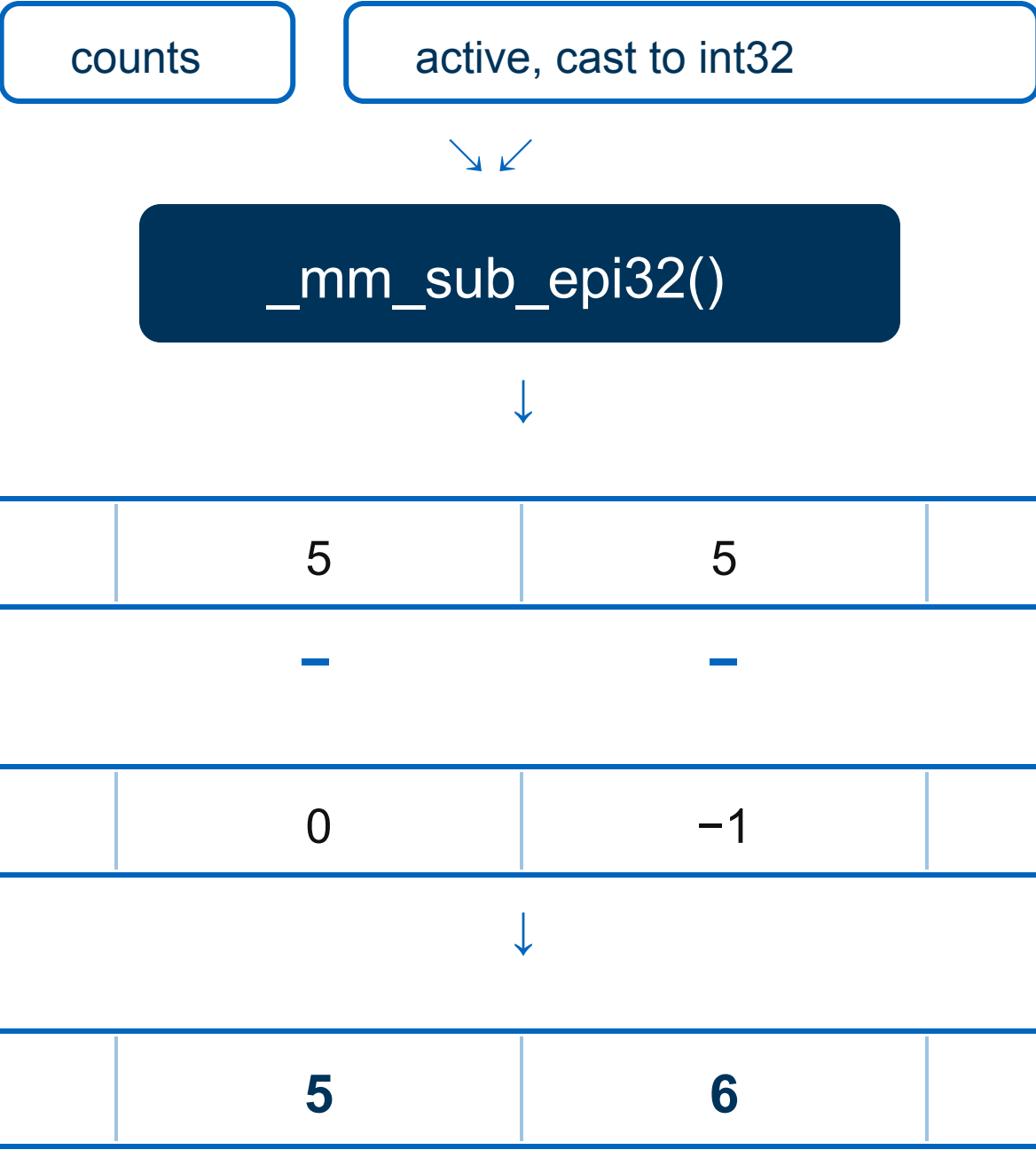
Lane 0	Lane 1	Lane 2	Lane 3
0xFFFFFFFF	0x00000000	0xFFFFFFFF	0x00000000
active	inactive	active	inactive

↓
_mm_castps_si128()

__m128i active

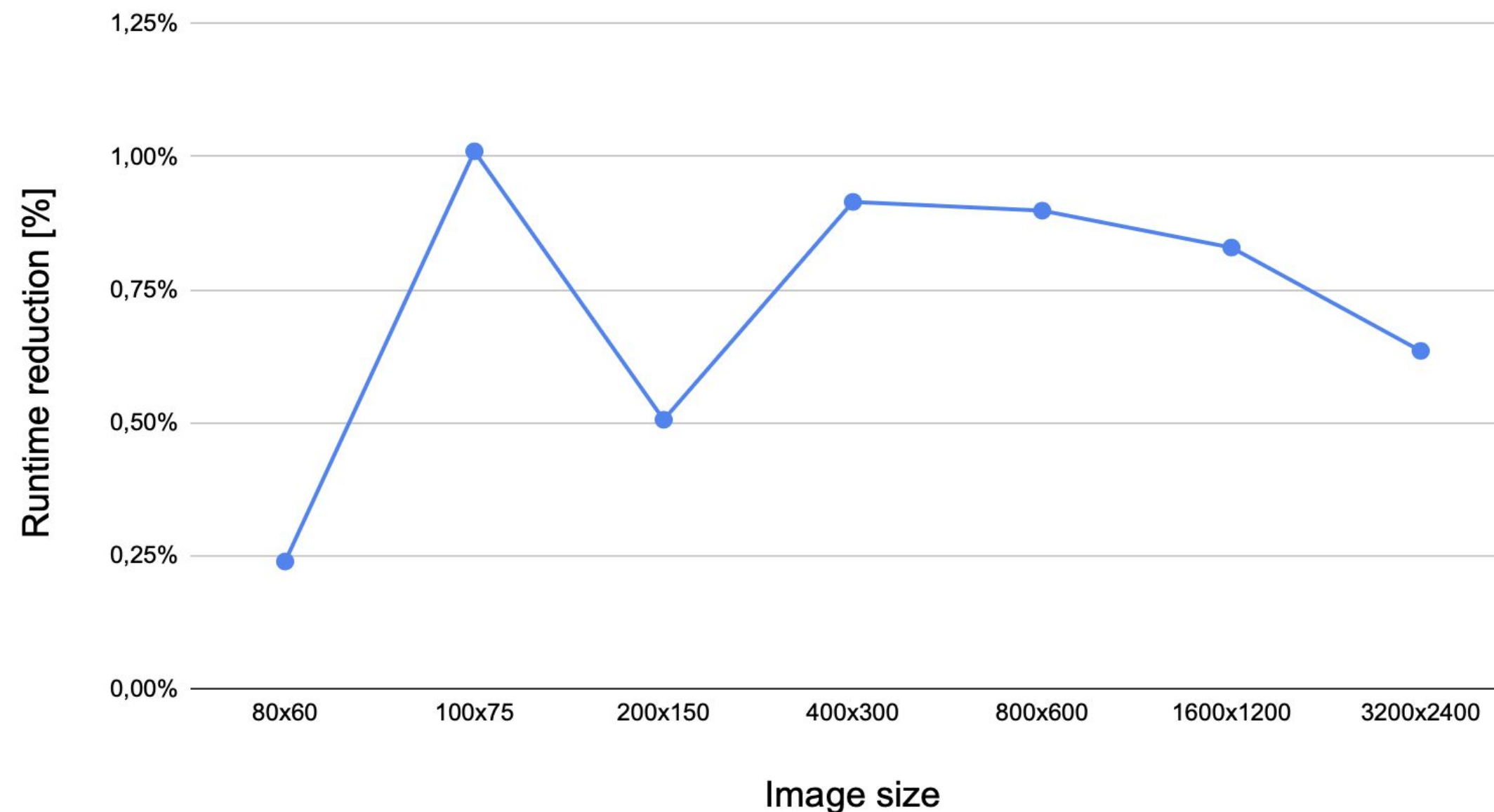
Lane 0	Lane 1	Lane 2	Lane 3
-1	0	-1	0
active	inactive	active	inactive

Simplified dependency graph



Optimized Counter Update: Measured Effect

Count-Mask Subtraction: Runtime Reduction



0.72%

mean runtime reduction

7 / 7

image sizes faster

0.24–1.01%

observed reduction range

Before: AND + ADD After: SUB

RETAINED

A very small but reproducible and positive measured effect, with one fewer vector operation and one fewer constant register.

Profiling Exposed the Per-Iteration Exit Chain

```
3.99      cmpleps    %xmm13,%xmm5
return __builtin_ia32_andps (__A, __B);
21.11     andps     %xmm5,%xmm3
return __builtin_ia32_movmskps ((__v4sf)__A);
20.25     movmskps  %xmm3,%r9d

// check to see if any haven't diverged yet
const __m128 inside = _mm_cmple_ps(magnitude_squared, four_v);
active = _mm_and_ps(active, inside); // 0x00000000

if (_mm_movemask_ps(active) == 0) { // 0x0000
0.52      test     %r9d,%r9d
25.01     ↑ jne     258
```

Required mask
update

Exit check on the
critical path
movmskps → test → jne

66.37%

of samples in the active-mask/exit
region

52.77% with color output

Every iteration

```
inside = _mm_cmple_ps(
    magnitude_squared, four_v);
```

active active inactive active

```
active = _mm_and_ps(active, inside);
```

Optimization candidate

```
if (_mm_movemask_ps(active) == 0)
    break;
```

SIMD mask → integer → branch
Every iteration → **Every K iterations**

Hypothesis: keep the active-mask update, but execute the movemask check less frequently.

K-Interval Exit Check: Checking Less Often

K defines how many Julia iterations are executed between the “all lanes inactive?” checks.

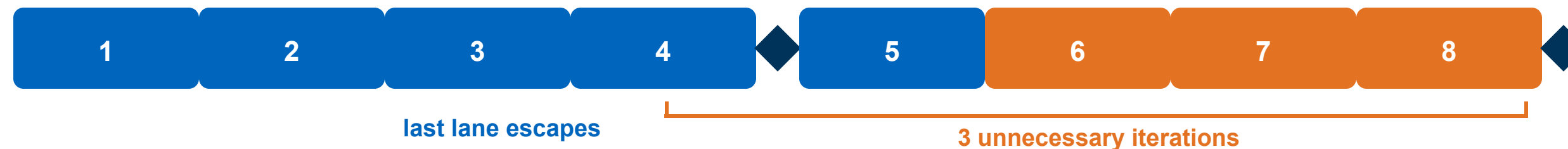
■ required SIMD iteration
 ■ unnecessary SIMD iteration
 ◆ movemask(active) == 0?

K = 1 — global exit test after every iteration



Iteration 5: last active lane escapes → next check: all lanes inactive → STOP

K = 4 — global exit test only after every four iterations



The next exit check does not occur until iteration 8.

Fewer global exit checks ↔ up to $K - 1$ unnecessary iterations

Core hypothesis

Fewer movemask checks could result in better performance.

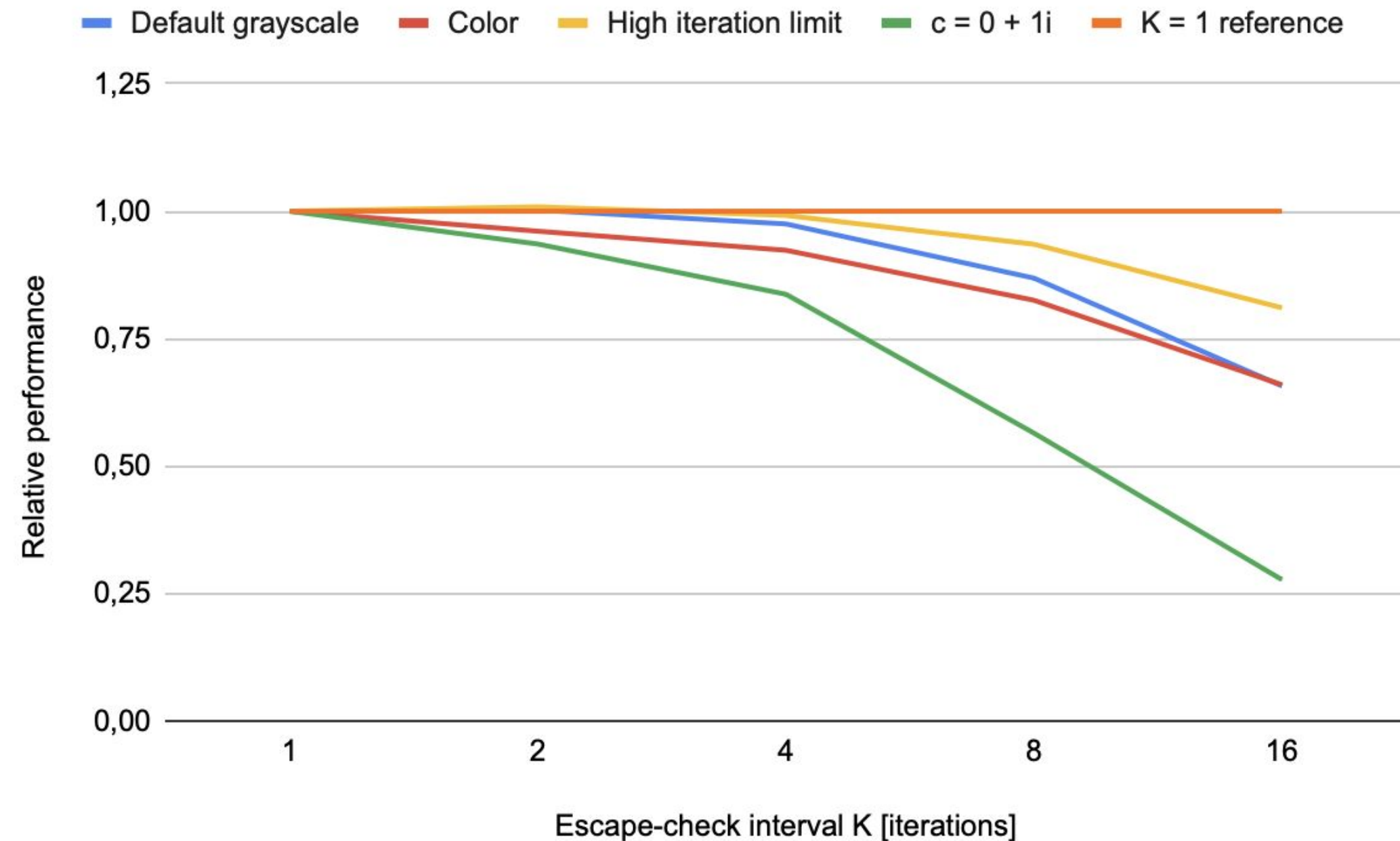
K = 1: check every iteration K = 4: check every four iterations

The per-lane escape comparison and active-mask update still execute every iteration. Only the global movemask-and-branch exit is deferred.

Correctness is preserved: inactive lanes remain masked and their counters no longer increase. Only loop termination is delayed.

The K-Sweep Rejected the Optimization

K-Interval: Performance Relative to K = 1



K = 2

Default grayscale:
1.002x · High iteration: 1.009x

K = 2

Color: 0.961x; $c = 0 + 1i$: 0.936x

K = 16

$c = 0 + 1i$: 0.277x

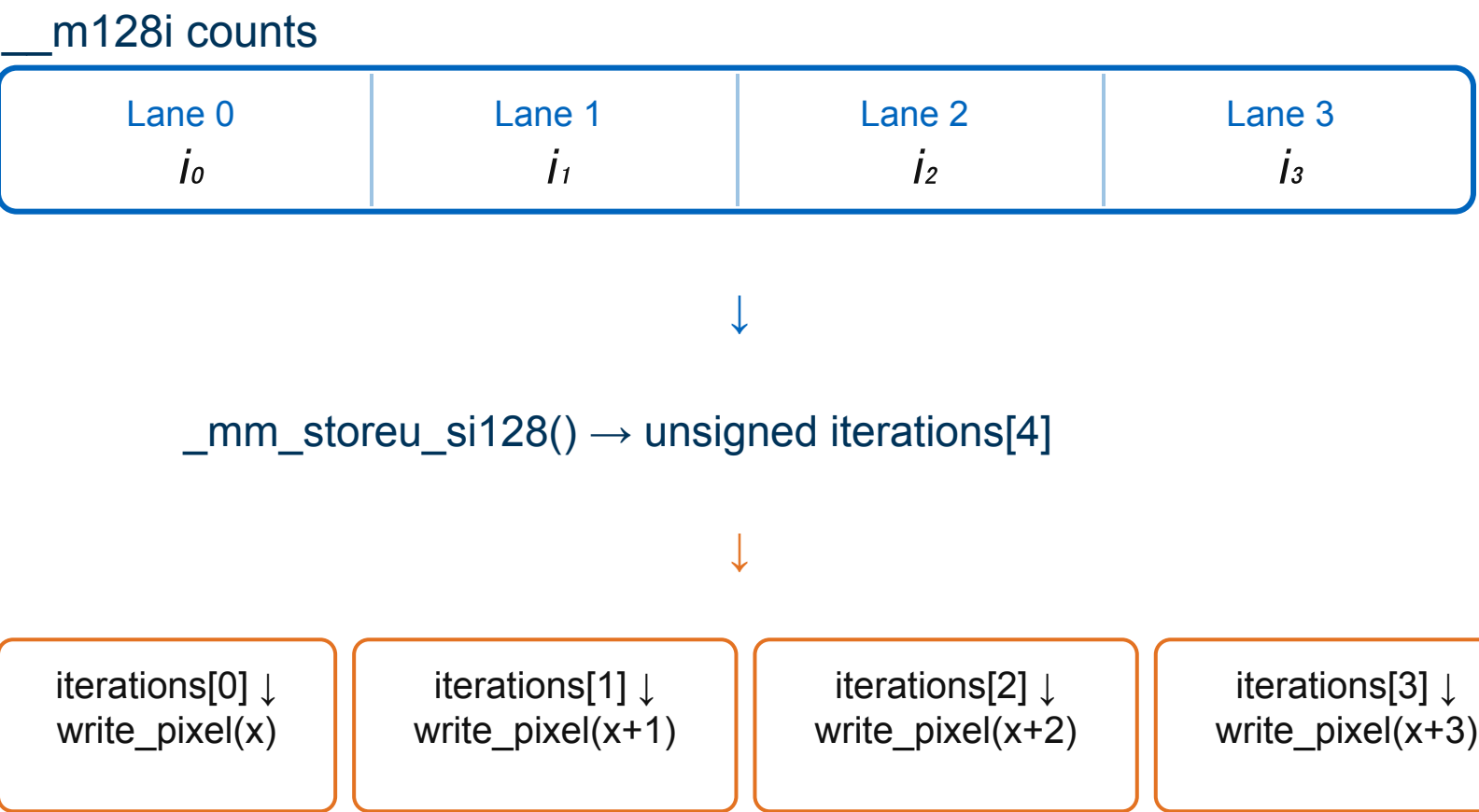
No $K > 1$ improved representative workloads.

REJECTED

No consistent gains and workload-dependent slowdowns caused by unnecessary masked iterations didn't justify keeping this change.

Go from scalar pixel write to SIMD

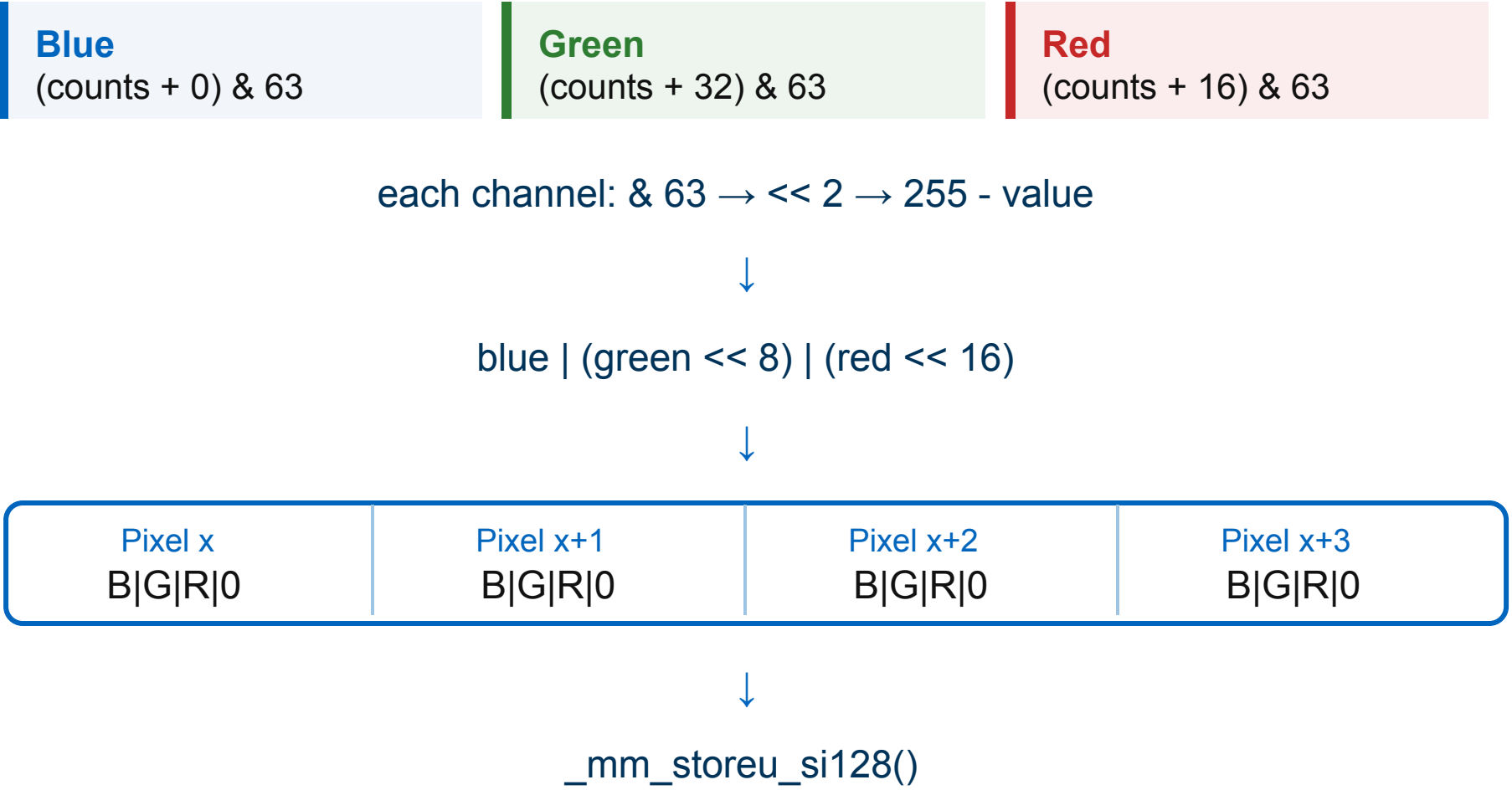
Before: SIMD → array → four scalar conversions



```
unsigned iterations[4];
_mm_storeu_si128((__m128i *) iterations, counts);

for (lane = 0; lane < 4; ++lane)
    write_pixel(...);
```

After: counts stay inside one SIMD register

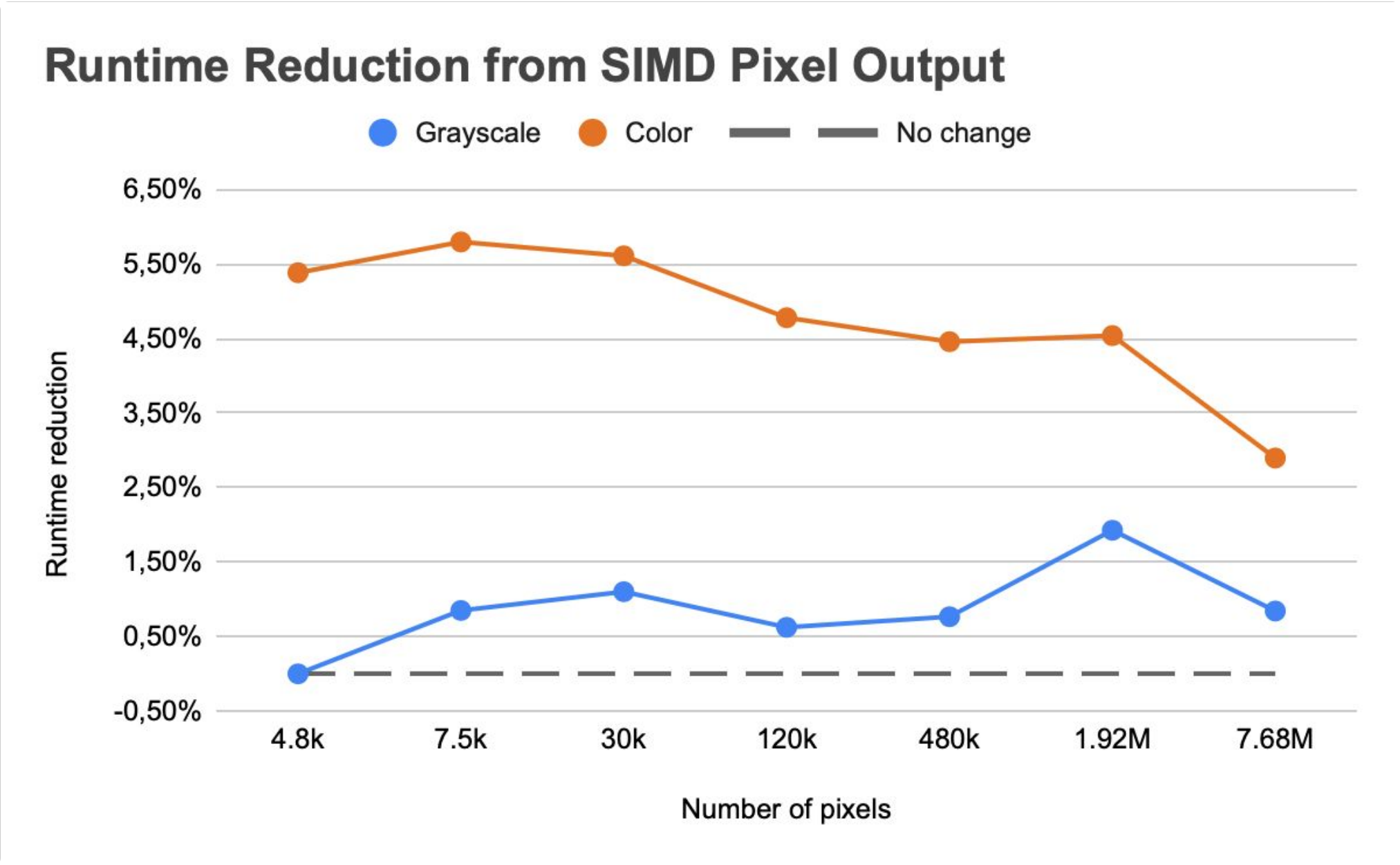


```
write_pixel4(row, x, counts, n, color);
```

24-bit
4 pixels = 12 bytes -> shuffle required

32-bit BGR0
4 pixels = 16 bytes -> one complete XMM register

Vectorized Output Primarily Benefits Color Images



Independent targeted 800 × 600 comparison

GRAYSCALE

4.354 ms → 4.316 ms **0.87% lower**

Total SIMD speedup: 3.04x → 3.07x

COLOR

4.652 ms → 4.420 ms **5.00% lower**

Total SIMD speedup: **2.87x** → **3.02x**

Total SIMD speedup — before vs after vectorized output

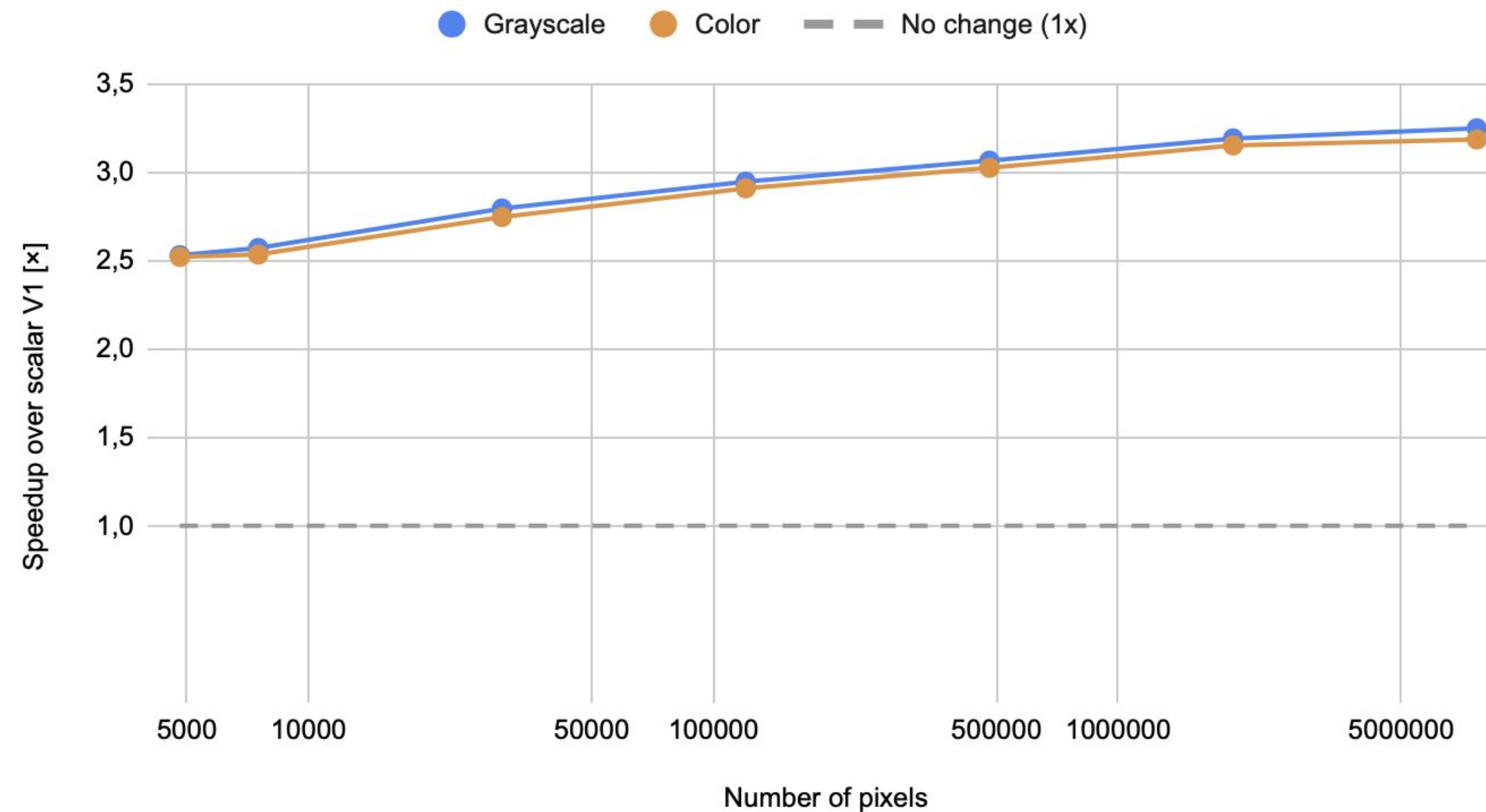
Before · Grayscale		3.04x
Before · Color		2.87x
After · Grayscale		3.07x
After · Color		3.02x

The gap between grayscale and color shrinks from **0.17x** to **0.05x**.

RETAINED Color conversion became a significant factor the Julia iteration was vectorized.

Final Speedup Across Image Sizes

Final SIMD Speedup Across Image Sizes



Speedup range

2.52x - 3.25x

Largest grayscale result

3.25x

Largest color result

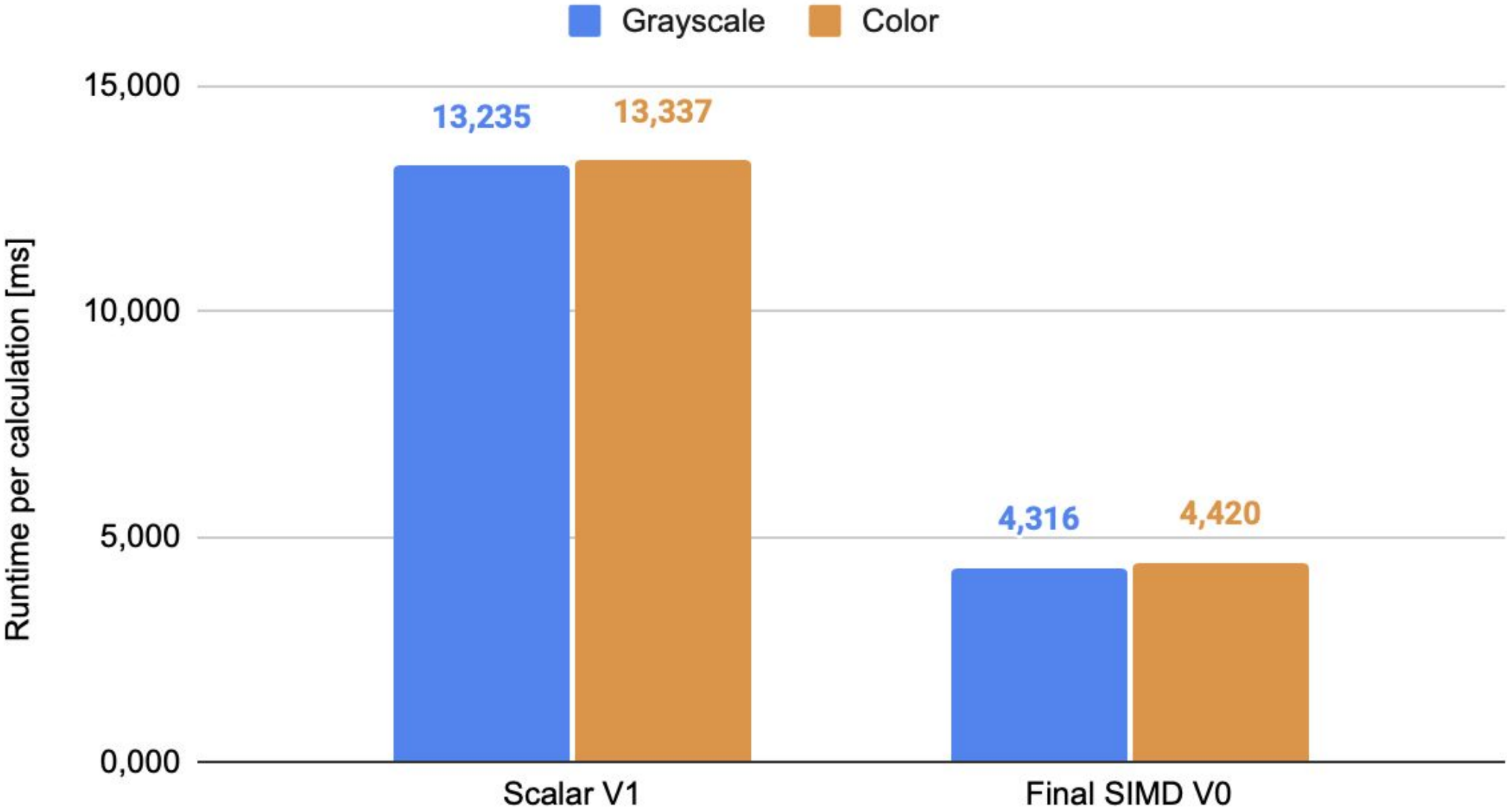
3.19x

Speedup increased with image size as fixed SIMD overhead becomes less significant.

Runtime at 800 x 600 Pixels



Runtime at 800 × 600 Pixels



Scalar V1 compared with final SIMD V0

Grayscale

13.235 ms → 4.316 ms

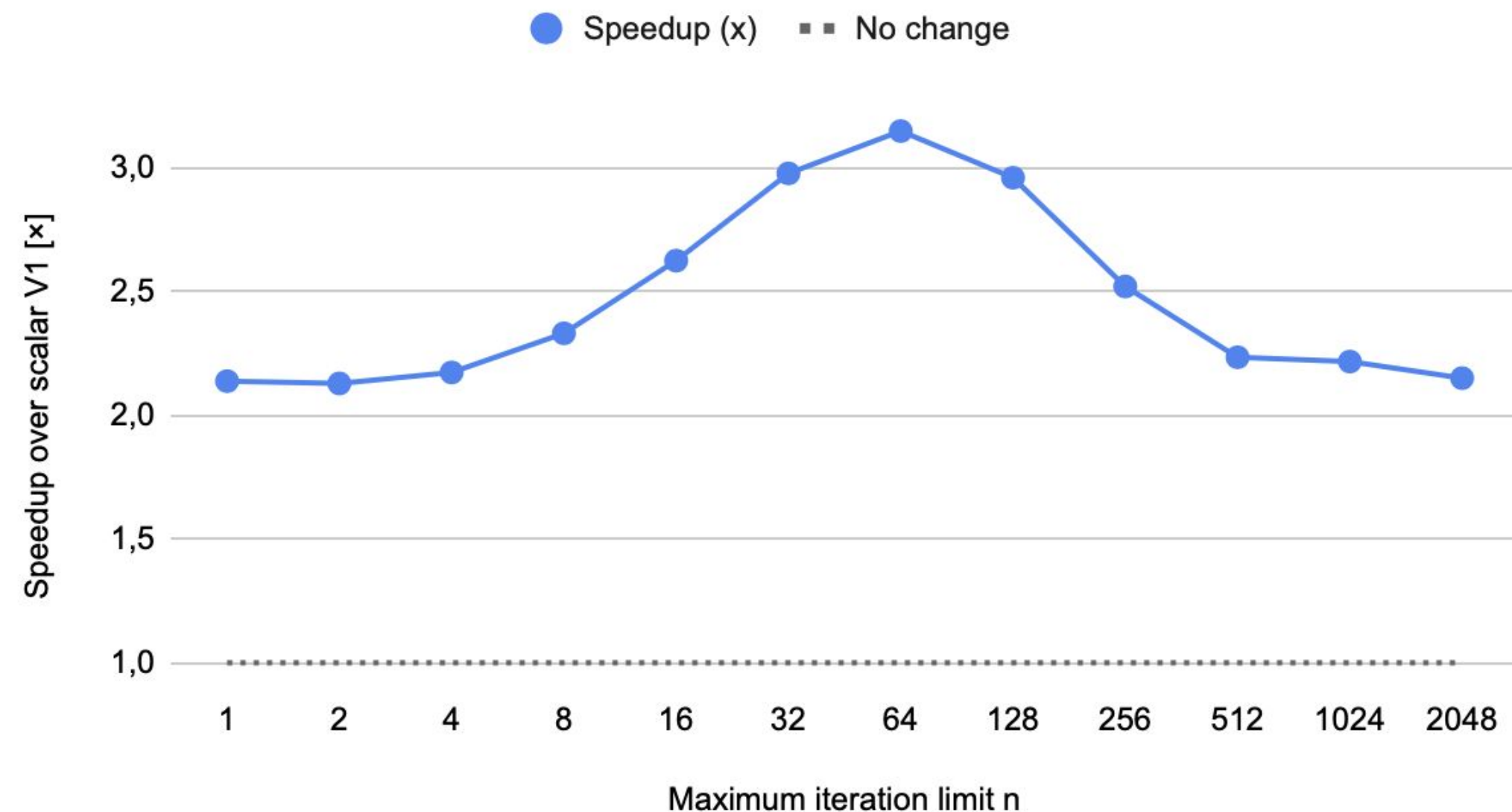
3.07x speedup

Color

13.337 ms → 4.420 ms

3.02x speedup

SIMD Speedup Depends on the Iteration Limit



Measured range

2.13x–3.15x

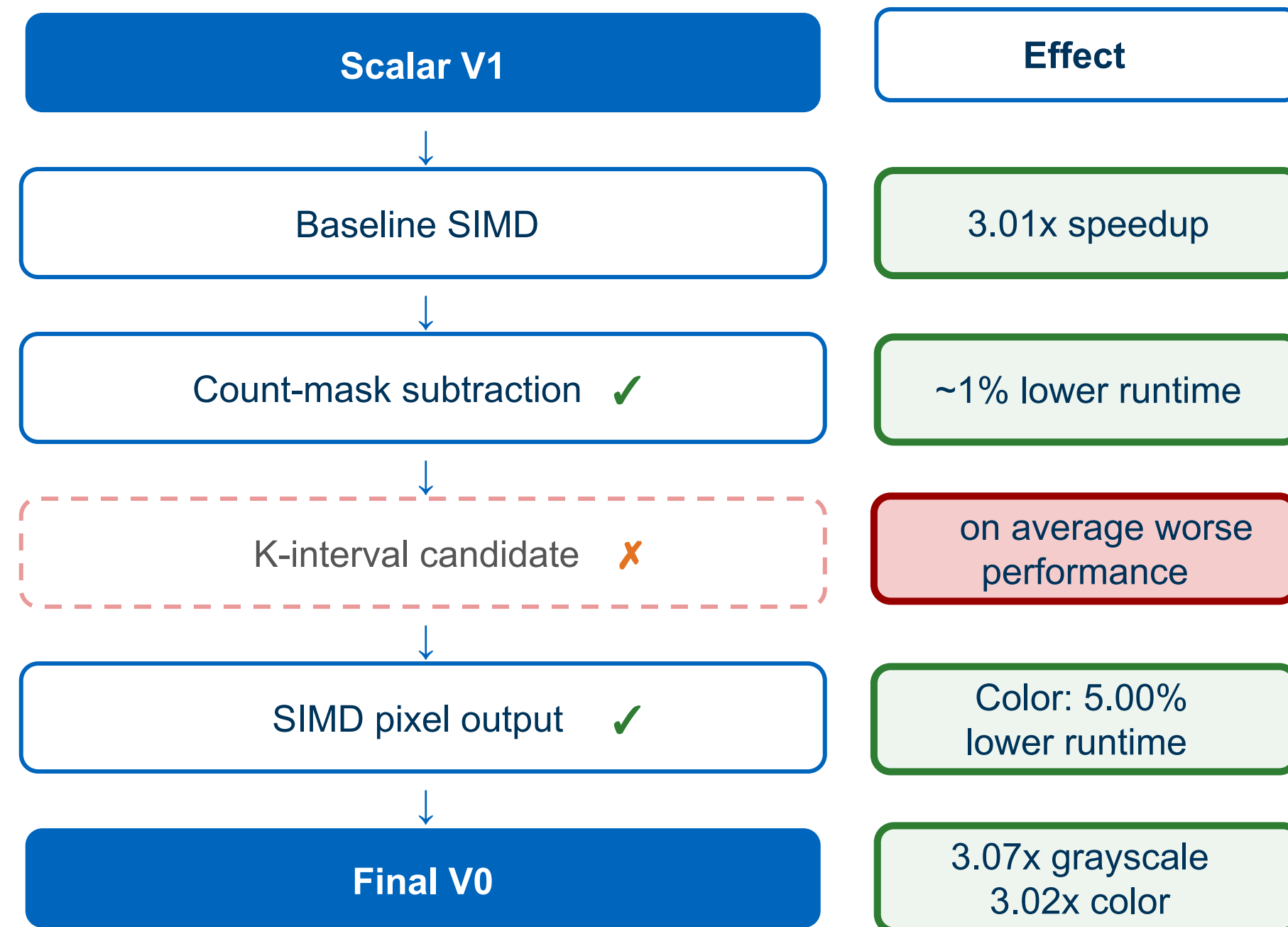
Peak result

3.15x at $n = 64$

Interpretation

- The SIMD overhead becomes more costly for small iteration limits
- Larger iteration limits often wait for just one pixel, resulting in lowered parallelization gains.

Summary and Outlook



Outlook

Use wider SIMD. The use of wider registers (AVX2/AVX512) that can handle more pixels calculations at once would result in easy gains

Swap out diverged pixels. Once a pixel in the register has diverged you could already swap this lane out with the next in the row.

Multithreading. Calculating one row of pixels per thread would result in further speedup -> limited by parallel SIMD capabilities of the used architecture